A Comparative Study of Three Algorithms for Mining Top-K High Utility Itemsets from Massive Datasets with Parallelism Analysis

Le Anh Tuan, To Hoang Dao

Ton Duc Thang University (TDTU), Ho Chi Minh City, Vietnam

Abstract. High Utility Itemset Mining (HUIM) is a fundamental task in data mining that discovers itemsets yielding high profit from transactional databases. Unlike traditional frequent itemset mining, HUIM considers both the quantity and unit profit of items, making it significantly more challenging due to the loss of the downward closure property. This paper presents a comparative study of three algorithms for mining top-k high utility itemsets: **TKU** (two-phase, UP-Tree-based), **TKO** (one-phase, utility-list-based), and **THUI** (one-phase, utility-list-based with LIU matrix). We analyze the theoretical foundations, map each algorithmic step to its corresponding Go implementation, and present experimental results comparing runtime, memory consumption, and scalability. Furthermore, we identify parallelization opportunities in each algorithm using techniques such as MapReduce and Fork/Join, and propose concrete strategies for improving performance on massive datasets.

Keywords: High Utility Itemset Mining · Top-K Mining · MapReduce · Fork/Join · Parallelism · SPMF · Massive Datasets

1 Introduction

Traditional frequent itemset mining (FIM) assumes all items have equal importance and uses a binary model (present or absent). However, in real-world retail analytics, items have different unit profits and appear in varying quantities. **High Utility Itemset Mining (HUIM)** [3] addresses this limitation by assigning external utilities (unit profit) and internal utilities (purchase quantity) to items.

A major challenge in HUIM is setting the minimum utility threshold (`min_util`). Setting it too low produces an overwhelming number of results; setting it too high yields none. The **top-k** formulation [1] elegantly solves this by asking the user to specify only the desired number of results k, while the algorithm automatically determines the appropriate threshold.

In this paper, we study and compare three representative algorithms:

1. **TKU** [1]: A two-phase algorithm using UP-Tree with five threshold-raising strategies (PE, NU, MD, MC, SE).

2. **TKO** [1]: A one-phase algorithm using utility-lists with strategies RUC, RUZ, and EPB.
3. **THUI** [2]: A one-phase algorithm using utility-lists enhanced with a Leaf Itemset Utility (LIU) matrix for more effective threshold raising.

All three algorithms aim to mine top-k high utility itemsets but employ fundamentally different data structures and pruning strategies. The remainder of this paper is organized as follows: Section 2 defines the problem formally. Section 3 details each algorithm with code-to-theory mapping. Section 4 presents experimental comparisons. Section 5 analyzes parallelization opportunities. Section 6 concludes.

2 Preliminaries and Problem Definition

2.1 Basic Definitions

Let $I = \{I_1, I_2, \ldots, I_m\}$ be a set of distinct items and $D = \{T_1, T_2, \ldots, T_n\}$ be a transactional database. Each transaction $T_r \in D$ is a subset of I. Each item I_j has an **external utility** (unit profit) $p(I_j)$ and an **internal utility** (quantity) $q(I_j, T_r)$ in transaction T_r.

Definition 1 (Utility of an Item). *The utility of item I_j in transaction T_r is:*

$$EU(I_j, T_r) = p(I_j) \times q(I_j, T_r) \tag{1}$$

Definition 2 (Utility of an Itemset). *The utility of an itemset X in the database is:*

$$EU(X) = \sum_{T_r \in D \wedge X \subseteq T_r} \sum_{I_j \in X} EU(I_j, T_r) \tag{2}$$

Definition 3 (Transaction Utility). *The transaction utility of T_r is:*

$$TU(T_r) = \sum_{I_j \in T_r} EU(I_j, T_r) \tag{3}$$

Definition 4 (Transaction-Weighted Utilization). *The TWU of an itemset X is:*

$$TWU(X) = \sum_{X \subseteq T_r \wedge T_r \in D} TU(T_r) \tag{4}$$

2.2 Key Properties

Theorem 1 (Transaction-Weighted Downward Closure – TWDC). *For any itemsets $X \subseteq Y$: $TWU(Y) \leq TWU(X)$. Furthermore, $EU(X) \leq TWU(X)$.*

This property is critical because, unlike $EU(X)$, $TWU(X)$ is anti-monotone. If $TWU(X) <$ `min_util`, then all supersets of X can be safely pruned.

Definition 5 (Minimum Item Utility – MIU).

$$MIU(X) = \left(\sum_{I_j \in X} \min_{T_r \supseteq X} \{EU(I_j, T_r)\} \right) \times SC(X) \tag{5}$$

*where $SC(X)$ is the support count of X. $MIU(X)$ is a **lower bound** of $EU(X)$, used to safely raise the threshold.*

Definition 6 (Utility-List). *For an itemset X, its utility-list $UL(X)$ contains one entry $(tid, iutil, rutil)$ per transaction T_r where $X \subseteq T_r$:*

- *tid: transaction identifier*
- *iutil $= EU(X, T_r)$: the exact utility of X in T_r*
- *rutil: sum of utilities of items ordered after X in T_r*

Key property: $EU(X) = \sum_{e \in UL(X)} e.iutil$ gives the exact utility directly.

2.3 Problem Statement

Top-k High Utility Itemset Mining: Given a transactional database D with utility information and an integer k, discover the k itemsets with the highest utility values without requiring a pre-defined minimum utility threshold.

3 Algorithm Analysis with Code-to-Theory Mapping

All three algorithms have been re-implemented in Go. We present each algorithm with pseudocode derived from the actual Go source code, with source file references (package path and line numbers) in each caption.

3.1 Algorithm 1: TKU (Two-Phase, Tree-Based)

TKU [1] operates in two phases. Phase 1 builds a compressed UP-Tree and generates candidate itemsets (PKHUIs) using five threshold-raising strategies. Phase 2 verifies candidates by scanning the original database to compute exact utilities.

Phase 1: Candidate Generation via UP-Tree

Step 1 – Database Scan and TWU Computation. The first database scan computes *TWU* for each item, and the minimum/maximum item utilities (*miu*, *mau*) needed by later pruning strategies.

Algorithm 1 TWU, MIU, MAU Computation — First DB Scan (Ref: `tku/tku.go`, `preEvaluation`, lines 216–269)

Require: Database D
Ensure: $TWU[\cdot]$, $MinUtil[\cdot]$, $MaxUtil[\cdot]$ for each item
for each transaction $T_r \in D$ **do**
  Parse T_r into items $\{I_1, \ldots, I_w\}$ and utilities $\{u_1, \ldots, u_w\}$
  $TU \leftarrow$ transaction utility of T_r
  for each item I_j in T_r with utility u_j **do**
    if $MinUtil[I_j] = 0$ **or** $MinUtil[I_j] > u_j$ **then**
      $MinUtil[I_j] \leftarrow u_j$ ▷ Track minimum item utility
    end if
    if $MaxUtil[I_j] < u_j$ **then**
      $MaxUtil[I_j] \leftarrow u_j$ ▷ Track maximum item utility
    end if
    $TWU[I_j] \leftarrow TWU[I_j] + TU$ ▷ Accumulate TWU
  end for
end for

Theory → Code: This implements Definition 4 (TWU) and collects $miu(I_j) = \min_{T_r}\{EU(I_j, T_r)\}$ used by strategies MD and MC.

Step 2 – Strategy PE (Pre-Evaluation). During the first scan, TKU also builds a Pre-Evaluation Matrix (PEM) using only the first item of each transaction paired with subsequent items:

$$PEM[I_1][I_i] \mathrel{+}= EU(\{I_1, I_i\}, T_r) \tag{6}$$

The k-th largest value in PEM safely raises `min_util_Border` from 0, since PEM values are lower bounds.

Algorithm 2 PE Strategy — Pre-Evaluation Matrix (Ref: `tku/tku.go`, `preEvaluation`+`getInitialUtility`, lines 216–311)

for each transaction $T_r = \{I_1, I_2, \ldots, I_w\}$ **do**
  for $i \leftarrow 2$ **to** w **do**
    $PEM[I_1][I_i] \leftarrow PEM[I_1][I_i] + EU(I_1, T_r) + EU(I_i, T_r)$
  end for
end for
$minUtil \leftarrow k$-th largest value in PEM

Step 3 – UP-Tree Construction. Transactions are filtered (items with $TWU <$ `min_util_Border` removed), sorted by TWU descending, and inserted into the UP-Tree. Each node stores: item name, count, and node utility (Nu). The UP-Tree compresses n transactions into shared-prefix paths, reducing memory from $O(n \cdot w)$ to $O(|\text{tree}|)$.

Step 4 – Strategy MD (MIU of Descendants). After the tree is built, TKU examines parent-descendant pairs to compute lower bounds:

$$MIU(\{\alpha, \beta\}) = (miu(\alpha) + miu(\beta)) \times SC_{\text{tree}}(\alpha, \beta) \tag{7}$$

Algorithm 3 MD Strategy — MIU of Descendants (Ref: `tku/tku.go`, `runPhase1CandidateGeneration`, lines 126–137)

for each child node c of *tree.root* **do**
  $SumDS[\cdot] \leftarrow$ count of each item among descendants of c
  $\alpha \leftarrow c.item$
  for each item β where $SumDS[\beta] \neq 0$ and $\beta \neq \alpha$ **do**
    $value \leftarrow (miu[\alpha] + miu[\beta]) \times SumDS[\beta]$
    UpdateTopKHeap($DSHeap, value$) ▷ Raise threshold
  end for
end for

Step 5 – UPGrowth with Strategy MC (MIU of Candidates). The recursive UP-Growth procedure traverses the tree FP-Growth-style. For each candidate X generated, MC computes $MIU(X)$ and inserts it into a size-k min-heap. When the heap is full, its minimum becomes the new threshold.

Algorithm 4 MC Strategy — MIU of Candidates (Ref: `tku/tku.go`, `updateNodeCountHeap`, lines 378–393)

Require: Candidate X with $SumMiu$, support count SC
$MIU \leftarrow SumMiu \times SC$
if $MIU > globalMinUtil$ **then**
  UpdateTopKHeap($CandidateHeap, MIU$)
end if

Safety guarantee: Since $MIU(X) \leq EU(X)$, raising the threshold using MIU values never misses true top-k results.

Phase 2: Exact Utility Verification with Strategy SE Phase 2 loads the entire original database into RAM as two parallel arrays (`HDB[]` for items, `BNF[]` for utilities), then verifies each candidate:

Algorithm 5 Phase 2 — Exact Utility Verification (Ref: `tku/phase2.go`, `readCandidateItemsets`, lines 125–193)

Require: Sorted candidates $\mathcal{C}$ (descending by estimated utility), database D
Ensure: Top-k HUIs with exact utilities

for each candidate $X \in \mathcal{C}$ **do**
  if $X.estUtil < minUtility$ **then**
    skip X ▷ SE: early termination
  end if
  $exactUtil \leftarrow 0$
  for each transaction $T_r \in D$ **do**
    if $X \subseteq T_r$ **then**
      $exactUtil \leftarrow exactUtil + EU(X, T_r)$ ▷ Sum exact utilities
    end if
  end for
  if $exactUtil \geq minUtility$ **then**
    Insert $(X, exactUtil)$ into Top-k Heap
    Update $minUtility$ from heap minimum
  end if
end for

Strategy SE: Candidates are sorted in descending order by estimated utility before verification. Strong candidates verified first rapidly raise $minUtility$, allowing weaker candidates to be skipped.

Complexity: Phase 2 is $O(C' \times N \times L)$ where C' is candidates verified, N transaction count, L average candidate length. This is TKU's main bottleneck.

3.2 Algorithm 2: TKO (One-Phase, Utility-List-Based)

TKO [1] eliminates Phase 2 by using **utility-lists** for direct exact utility computation during mining.

Utility-List Construction and Join To mine itemset $X_{ab} = X_a \cup X_b$, TKO joins the utility-lists of X_a and X_b:

Algorithm 6 Utility-List Join for TKO

Require: Utility-lists $UL(X_a)$, $UL(X_b)$, prefix $UL(P)$
Ensure: Utility-list $UL(X_{ab})$
for each entry $e_a \in UL(X_a)$ **do**
  if $\exists e_b \in UL(X_b)$ s.t. $e_a.tid = e_b.tid$ **then**
    if $P \neq \emptyset$ **then**
      Find $e_p \in UL(P)$ s.t. $e_p.tid = e_a.tid$
      $iutil \leftarrow e_a.iutil + e_b.iutil - e_p.iutil$
    else
      $iutil \leftarrow e_a.iutil + e_b.iutil$
    end if
    Add $(e_a.tid, iutil, e_b.rutil)$ to $UL(X_{ab})$
  end if
end for

Key advantage: $EU(X_{ab}) = \sum_{e \in UL(X_{ab})} e.iutil$ — exact utility is available immediately, no second database scan needed.

Pruning Strategies

RUC (Raising by Utility of Candidates). Whenever $EU(X) \geq$ `min_util_Border`, insert X into a size-k min-heap. When full, raise the threshold to the heap's minimum.

RUZ (Reducing by Z-elements). If extension item z has $rutil = 0$ in all utility-list entries, the upper bound $\sum(iutil + rutil)$ is tightened, enabling stronger pruning.

EPB (Exploring Promising Branches first). Extensions are sorted by $\sum(iutil + rutil)$ descending. By exploring strongest branches first, TKO discovers high-utility itemsets early and raises the threshold faster.

Pruning condition: If $\sum_{e \in UL(X)}(e.iutil + e.rutil) <$ `min_util_Border`, prune all supersets of X.

3.3 Algorithm 3: THUI (One-Phase, Utility-List + LIU Matrix)

THUI [2] is a one-phase algorithm that, like TKO, uses utility-lists, but introduces a novel **Leaf Itemset Utility (LIU)** matrix to achieve more aggressive threshold raising, particularly by leveraging information about leaf-level (longest) itemsets.

Overview and Key Innovation THUI's main contribution is that it computes utility information for **leaf itemsets** — itemsets corresponding to complete transaction suffixes in the search tree — during the database scan phase. This yields tighter lower bounds for the threshold *before* the recursive mining even begins.

Step 1 – RIU: Raising by Real Item Utilities During the first scan, THUI computes the **real item utility** (RIU) for each single item:

$$RIU(I_j) = \sum_{T_r \supseteq \{I_j\}} EU(I_j, T_r) \tag{8}$$

The k-th largest RIU value is used to raise the initial threshold.

Algorithm 7 RIU Computation — First DB Scan (Ref: `thui/thui.go`, `firstScan`, lines 315–352)

Require: Database D, parameter k
Ensure: Initial $minUtility$ raised by RIU
for each transaction $T_r \in D$ **do**
  $TU \leftarrow$ transaction utility of T_r
  for each item I_j in T_r with utility u_j **do**
    $TWU[I_j] \leftarrow TWU[I_j] + TU$ ▷ TWU accumulation
    $RIU[I_j] \leftarrow RIU[I_j] + u_j$ ▷ Real item utility
  end for
end for
Sort items by RIU in descending order
if number of items $\geq k$ **then**
  $minUtility \leftarrow k$-th largest RIU value
end if

Theory → Code: Since $RIU(I_j) = EU(\{I_j\})$ is the exact utility of a single-item set, and any top-k result has $EU \geq$ the k-th largest single-item utility, this is a safe lower bound.

Step 2 – Utility-List Construction with LIU Matrix During the second scan, THUI builds utility-lists identically to TKO/HUI-Miner. Additionally, it constructs two auxiliary structures:

EUCS Map (optional). For each pair of items (I_a, I_b), stores accumulated TWU and real 2-item utility. Used similarly to TKO's EUCS pruning.

Algorithm 8 EUCS Pairwise Utility Update (Ref: `thui/thui.go`, `updateEUCSprune`, lines 439–458)

Require: Current item pair (I_a, I_b) in revised transaction, $newTWU$
for each item I_b after I_a in the revised transaction **do**
  $EUCS[I_a][I_b].twu \leftarrow EUCS[I_a][I_b].twu + newTWU$
  $EUCS[I_a][I_b].utility \leftarrow EUCS[I_a][I_b].utility + u_a + u_b$
end for

LIU Matrix (Leaf Itemset Utility). The core innovation of THUI. For each item at the end of a transaction ("leaf"), THUI tracks the cumulative utility of complete suffix sequences:

Algorithm 9 LIU Matrix Update — Leaf Itemset Utility (Ref: `thui/thui.go`, `updateLeafprune`, lines 460–480)

Require: Item I_{leaf} at position i, revised transaction T', utility-lists ULs
$cutil \leftarrow u_{leaf}$ ▷ Cumulative utility starts from leaf
$leafIdx \leftarrow$ index of I_{leaf} in ULs (sorted by TWU)
for $j \leftarrow i-1$ **downto** 0 **do** ▷ Walk backwards
  $I_{prev} \leftarrow T_j.item$
  $prevIdx \leftarrow leafIdx - 1$
  if $ULs_{prevIdx}.item \neq I_{prev}$ **then**
    break ▷ Not consecutive in sort order
  end if
  $cutil \leftarrow cutil + u_{prev}$
  $LIU[leafIdx][prevIdx] \leftarrow LIU[leafIdx][prevIdx] + cutil$
  $leafIdx \leftarrow prevIdx$
end for

Theory: The LIU matrix captures the utility of *complete consecutive suffix itemsets* in transactions. For a transaction $T = \{a, b, c, d, e\}$ (sorted by TWU), the leaf item is e, and consecutive suffixes like $\{d, e\}$, $\{c, d, e\}$ are tracked. Their accumulated utilities across all transactions provide tight lower bounds.

Step 3 – Threshold Raising via LIU-E and LIU-LB Before recursive mining begins, THUI uses the LIU matrix to raise the threshold with two sub-strategies:

LIU-Exact (LIU-E). Exact utility values stored in the LIU matrix are directly inserted into a size-k priority queue:

Algorithm 10 LIU-Exact Threshold Raising (Ref: `thui/thui.go`, `raisingThresholdLeaf`, lines 686–692)

for each entry $(leafIdx, startIdx) \rightarrow value$ in LIU **do**
  if $value \geq minUtility$ **then**
    InsertTopKHeap($leafHeap, value$) ▷ Raise threshold
  end if
end for
if $|leafHeap| \geq k$ and $leafHeap.min > minUtility$ **then**
  $minUtility \leftarrow leafHeap.min$
end if

LIU-Lower Bound (LIU-LB). THUI generates combinatorial lower bounds by subtracting subsets of items from known leaf utilities:

Algorithm 11 LIU-LB Threshold Raising (Ref: `thui/thui.go`, `raisingThresholdLeaf`, lines 694–721)

Require: Leaf entry with range $[st, end)$ and total utility V

for $i \leftarrow st + 1$ **to** $end - 2$ **do** ▷ Remove one item
  $V_1 \leftarrow V - EU(ULs[i])$
  if $V_1 \geq minUtility$ **then**
    InsertTopKHeap($leafHeap, V_1$)
  end if
  for $j \leftarrow i + 1$ **to** $end - 2$ **do** ▷ Remove two items
    $V_2 \leftarrow V - EU(ULs[i]) - EU(ULs[j])$
    if $V_2 \geq minUtility$ **then**
      InsertTopKHeap($leafHeap, V_2$)
    end if
  end for
end for

Theory: If a leaf itemset $\{c, d, e\}$ has accumulated utility V, then the subset $\{c, e\}$ (removing d) has utility $\geq V - EU(\{d\})$. This provides additional lower bounds without any database scanning.

Step 4 – Recursive Mining (THUI procedure) The core mining is structurally identical to HUI-Miner/TKO — recursive utility-list joins with on-the-fly threshold raising:

Algorithm 12 THUI Recursive Mining (Ref: `thui/thui.go`, `thui`, lines 488–522)

Require: Prefix P, prefix utility-list pUL, extension utility-lists ULs

for $i \leftarrow |ULs| - 1$ **downto** 0 **do** ▷ Check single extensions
  if $ULs_i.sumIutils \geq minUtility$ **then**
    SaveToTopK($P \cup \{ULs_i.item\}$, ULs_i)
  end if
end for
for $i \leftarrow |ULs| - 2$ **downto** 0 **do** ▷ Generate multi-item extensions
  $X \leftarrow ULs_i$
  if $X.sumIutils + X.sumRutils \geq minUtility$ **then** ▷ Pruning
    $exULs \leftarrow \emptyset$
    for $j \leftarrow i + 1$ **to** $|ULs| - 1$ **do**
      $Y \leftarrow ULs[j]$
      $Z \leftarrow$ Construct(pUL, X, Y) ▷ Join utility-lists
      if $Z \neq null$ **then**
        $exULs \leftarrow exULs \cup \{Z\}$
      end if
    end for
    THUI-Mine($P \cup \{X.item\}, X, exULs$) ▷ Recurse
  end if
end for

The SaveToTopK method maintains a size-k min-heap and raises $minUtility$ whenever the heap is full (Ref: `thui/thui.go`, `save`, lines 594–614).

4 Experimental Comparison

4.1 Experimental Setup

All three algorithms (TKU, TKO, THUI) are implemented in Go. Experiments were conducted on a machine with Apple M-series CPU, 16GB RAM, running macOS. We used the standard SPMF input format: `items:TU:utilities`.

4.2 Datasets

Table 1. Dataset characteristics

Dataset	$\|D\|$	$\|I\|$	Avg Len	Type	Source
DB_Utility	5	7	4.4	Synthetic	SPMF
Retail	88,162	16,470	10.3	Sparse	FIMI
Mushroom	8,124	119	23.0	Dense	UCI
Chess	3,196	75	37.0	Dense	FIMI
Accidents	340,183	468	33.8	Dense	FIMI
Kosarak	990,002	41,270	8.1	Sparse	FIMI

Datasets are available from the SPMF repository [4].

4.3 Comparison Results

Table 2. Runtime comparison (seconds, $k = 1000$)

Dataset	TKU	TKO	THUI
DB_Utility	0.01	0.01	0.01
Retail	2.45	1.12	**0.85**
Mushroom	15.8	4.20	**2.80**
Chess	52.3	8.90	**5.60**
Accidents	120+	35.2	**18.5**
Kosarak	8.50	3.80	**2.10**

Table 3. Peak memory usage (MB, $k = 1000$)

Dataset	TKU	TKO	THUI
DB_Utility	5	5	5
Retail	180	250	280
Mushroom	450	800	650
Chess	350	1200	1100
Accidents	2500	3500	3000
Kosarak	500	350	400

4.4 Analysis

TKU is the slowest due to its inherent two-phase design. Phase 2 requires scanning the entire database for each candidate, with complexity $O(C' \times N \times L)$. However, TKU has the lowest memory usage on dense datasets because the UP-Tree compresses data effectively and Phase 2 does not maintain utility-lists.

TKO eliminates Phase 2 entirely, computing exact utilities via utility-list joins. However, on dense datasets (Chess, Accidents), the number of utility-list entries explodes, consuming significant memory. TKO's strategies (RUC, RUZ, EPB) are effective but raise the threshold only during mining.

THUI achieves the best runtime by aggressively raising the threshold *before* mining begins. The LIU-E strategy provides exact leaf utility values, while LIU-LB generates combinatorial lower bounds from subsets. Together, they raise $minUtility$ significantly higher than TKO's starting point, pruning more branches

in the recursive search. THUI's memory overhead is slightly higher than TKO due to the LIU matrix, but the runtime improvement more than compensates.

Table 4. Summary of algorithm characteristics

Feature	**TKU**	**TKO**	**THUI**
Phases	2	1	1
Data structure	UP-Tree	Utility-List	Utility-List + LIU
DB scans	3+ (2 build + N verify)	2	2
Exact utility	Phase 2 only	During mining	During mining
Threshold strategies	PE, NU, MD, MC, SE	RUC, RUZ, EPB	RIU, EUCS, LIU-E, LIU-LB
Initial threshold	Moderate (PE)	0 (or low)	High (RIU + LIU)
Memory	Low–Moderate	High (dense)	High (dense) + LIU
Best for	Small–medium data	Sparse data	Dense & large data

5 Parallelization Opportunities

5.1 Parallelism Models

MapReduce [5] is a distributed data-parallel model: data is partitioned across nodes, mappers process partitions independently, reducers aggregate results. Suitable for embarrassingly parallel tasks like database scanning.

Fork/Join [6] is a task-parallel model on a single multi-core machine. Tasks are recursively divided (fork) and results merged (join). Go's goroutine scheduler implements work-stealing natively.

5.2 Parallelizing TKU

Phase 1: TWU Computation & Tree Mining

Algorithm 13 MapReduce for TWU Computation

1: **Map**(T_r): **for each** $I_j \in T_r$ **do** emit(I_j, $TU(T_r)$)
2: **Reduce**(I_j, $[v_1, v_2, \ldots]$): $TWU(I_j) \leftarrow \sum v_i$

TWU Computation (MapReduce). With P partitions, each mapper processes N/P transactions — linear speedup.

UPGrowth Mining (Fork/Join). Each item in the header table produces an independent branch. Branches can be processed in parallel by forking each into a separate thread/goroutine:

Algorithm 14 Parallel UPGrowth via Fork/Join

for each item I_j in header table **do** ▷ Independent branches
  fork → ProcessBranch($I_j, DB_{imm}, localState_j, topK$)
end for
join all ▷ Wait for all branches to complete

Immutability for safety. The database and TWU arrays are **immutable** after construction — safe to share across threads. Only the top-k heap is mutable shared state, protected with a read-write lock and double-check pattern:

Algorithm 15 Thread-Safe Top-k Threshold Update (Double-Check Lock)

Require: New utility value u
acquire read-lock
if $u \leq minUtil$ **then**
  release read-lock; **return** false ▷ Fast rejection
end if
release read-lock
acquire write-lock
if $u \leq minUtil$ **then**
  release write-lock; **return** false ▷ Double-check
end if
Insert u into heap; update $minUtil$
release write-lock
return true

Phase 2: Candidate Verification (MapReduce) Phase 2 is the most parallelizable step:

Algorithm 16 MapReduce for Phase 2 Verification

Map($partition_i$, $candidates$):
for each $T_r \in partition_i$, each candidate X **do**
  if $X \subseteq T_r$ **then**
    emit(X, $EU(X, T_r)$)
  end if
end for
Reduce(X, $[u_1, \ldots]$): $EU(X) \leftarrow \sum u_i$

When DB exceeds RAM: Use streaming batch — scan DB once, verify all candidates simultaneously per chunk:

Algorithm 17 Streaming Batch Verification (Out-of-Core)

for each transaction T_r read from disk (streaming) **do**
  for each candidate $X \in \mathcal{C}$ **do** ▷ Verify ALL at once
    if $X \subseteq T_r$ **then**
      $exactUtil[X] \leftarrow exactUtil[X] + EU(X, T_r)$
    end if
  end for
end for

5.3 Parallelizing TKO

Utility-List Construction (Fork/Join). Initial utility-lists for single items can be built in parallel by partitioning the database, then merging partial lists.

Recursive Search (Fork/Join). Each extension branch in the recursive search is independent and can be forked. Work-stealing schedulers naturally handle unbalanced branches.

Challenge. Joining utility-lists requires the parent's list, creating dependencies that limit parallelism at deeper recursion levels.

5.4 Parallelizing THUI

RIU Computation (MapReduce). Like TWU, RIU computation is a simple sum-aggregation per item — trivially parallelizable.

LIU Matrix Construction (MapReduce). Each transaction contributes independently to the LIU matrix. Mappers process partitions and emit (leafIndex, suffixIndex, utility); reducers sum:

Algorithm 18 MapReduce for LIU Matrix

Map($partition_i$):
for each $T_r \in partition_i$ **do**
  for each leaf-suffix pair $(leaf, start, cutil)$ **do**
    emit($(leaf, start)$, $cutil$)
  end for
end for
Reduce($(leaf, start)$, $[c_1, \ldots]$): sum $\leftarrow \sum c_i$

LIU-LB Threshold Raising (Fork/Join). The nested loop in LIU-LB (Algorithm 11) iterates over combinatorial subsets. Each outer iteration is independent and can be forked into separate threads.

Recursive Mining (Fork/Join). Identical to TKO — the THUI mining procedure's extension loop (Algorithm 12) can be parallelized by forking each extension branch.

5.5 Summary of Parallelization Opportunities

Table 5. Parallelization opportunities per algorithm

Step	**TKU**	**TKO**	**THUI**
DB Scan / TWU	MapReduce	MapReduce	MapReduce
Threshold pre-raise	PE: MapReduce	—	RIU: MapReduce LIU: MapReduce LIU-LB: Fork/Join
Tree/List build	Pipeline	Fork/Join	Fork/Join
Recursive mining	Fork/Join	Fork/Join	Fork/Join
Candidate verify	MapReduce	N/A	N/A
Shared mutable state	TopK Heap	TopK Heap	TopK Heap
Sync mechanism	RWMutex	RWMutex	RWMutex

Table 6. Best parallelism model by scenario

Scenario	**Model**	**Rationale**
Single machine, DB fits RAM	Fork/Join	Low overhead, work stealing
Single machine, DB > RAM	Streaming + goroutines	Memory-efficient
Cluster, DB distributed	MapReduce (Spark)	Horizontal scaling
Deep recursive search	Fork/Join + semaphore	Limit goroutine explosion

6 Conclusion

We compared three algorithms for mining top-k high utility itemsets:

1. **TKU** is the simplest conceptually but suffers from two-phase overhead. Its Phase 2 brute-force verification is the main bottleneck ($O(C' \times N \times L)$). However, Phase 1 strategies (PE, NU, MD, MC) are elegant.
2. **TKO** eliminates Phase 2 using utility-lists for exact utility computation. It trades fewer database scans for higher memory on dense datasets. Strategies RUC, RUZ, EPB work synergistically.

3. **THUI** achieves the best runtime by raising the threshold aggressively *before* mining via LIU-E and LIU-LB. The LIU matrix captures leaf itemset utilities during the database scan, providing tight lower bounds that prune significantly more branches than TKO's initial threshold of 0.

For parallelization:

- Database scanning and TWU/RIU computation are embarrassingly parallel via **MapReduce**.
- Recursive mining (UPGrowth, TKO search, THUI search) fits **Fork/Join** with work-stealing.
- The shared mutable threshold requires read-write locking with double-check pattern. Keeping the database **immutable** and mining state **local per thread** is the key design principle.
- THUI's LIU-LB combinatorial loop and TKU's Phase 2 offer the richest parallelization opportunities.

Our Go re-implementation demonstrates that modern languages with built-in concurrency primitives provide efficient frameworks for parallelizing these algorithms without heavyweight distributed systems.

References

1. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient Algorithms for Mining Top-K High Utility Itemsets. IEEE Trans. Knowl. Data Eng. **28**(1), 54–67 (2016). https://doi.org/10.1109/TKDE.2015.2458860
2. Krishnamoorthy, S.: Mining Top-K High Utility Itemsets with Effective Threshold Raising Strategies. Expert Syst. Appl. **117**, 148–165 (2019). https://doi.org/10.1016/j.eswa.2018.09.051
3. Fournier-Viger, P., Lin, J.C.W., Vo, B., Chi, T.T., Zhang, J., Le, H.B.: A Survey of High Utility Itemset Mining. In: High-Utility Pattern Mining. Springer, Cham (2019)
4. Fournier-Viger, P., et al.: The SPMF Open-Source Data Mining Library Version 2. In: Proc. ECML-PKDD (2016). http://www.philippe-fournier-viger.com/spmf/
5. Dean, J., Ghemawat, S.: MapReduce: Simplified Data Processing on Large Clusters. Commun. ACM **51**(1), 107–113 (2008)
6. Lea, D.: A Java Fork/Join Framework. In: Proc. ACM Java Grande Conf., pp. 36–43 (2000)
7. Tseng, V.S., Wu, C.W., Shie, B.E., Yu, P.S.: UP-Growth: An Efficient Algorithm for High Utility Itemset Mining. In: Proc. SIGKDD, pp. 253–262 (2010)
8. Liu, M., Qu, J.: Mining High Utility Itemsets without Candidate Generation. In: Proc. CIKM, pp. 55–64 (2012)